

The Imposed and Emergent Pieces of Convolution Under an Energy Budget

Vasili Gavrilov*

First version 21 July 2026; revised 2 September 2026

Abstract

Which pieces of a convolution must be *imposed* as priors, and which *emerge* on their own? We grow a network’s topology from an exhaustive, fully-connected seed under an evolutionary energy budget that pays for every connection, using the search as a *lens* rather than an optimizer, and map the boundary. Sparse task-relevant connectivity emerges cleanly, and weight sharing is *adopted* when translation invariance rewards it. The compact aligned local filter emerges only once mutation acts on the shared feature the way real genetics does, rather than on a single edge, and composition on the channel axis does not emerge from the task label alone; in two dimensions that axis meets a capacity wall at three operations which four interventions fail to move. On top of the priors a ConvNet hardwires (locality, translatability), the free dimensions organize to the task: compact fields, and a depth that grows with the receptive field while overshooting it.

The contribution is the decomposition, and one mechanism it turns up. A connection cost as a lens on emergent structure has precedent; we turn it on *convolution*, and report a mechanism that does not follow from that precedent (a compact filter emerges only when mutation acts on the whole shared feature, not on a single edge), a map of which pieces are imposed and which emerge with the negatives kept, including the measured boundary where composition needs supervision, and a control showing that the directed search does real work: at a matched budget it finds a tidier, more energy-efficient solution than random search, and a modestly more accurate one. The rest reproduces textbook inductive-bias facts, verified against matched baselines. Every comparison is paired by seed and tested (Wilcoxon signed-rank or McNemar’s exact test, Holm-corrected); each experiment states its seed count and regenerates from a named probe in the reference implementation.

1 Introduction

This paper grows a network’s architecture instead of searching for it (Figure 1): start from an exhaustive, fully-connected seed, put a cost on every connection, and see which parts of convolution survive. The question is not “can a search find a good architecture” but “what structure emerges on its own under an energy budget, and what does not,” in the best case rediscovering the convolutional receptive field.

The idea has a long history. A 1997 thesis argued that heuristics built into the architecture by hand are a *necessity* for a trainable network [25]; the question here is the complement, whether that structure can instead *emerge* from the search. An earlier unpublished companion study came at the same idea from the search side and reached a firm negative: on NAS-Bench-101 no crossover meaningfully beats random search, because good cells are common and the gap to the optimum sits inside the benchmark’s noise. Section 2.4 returns to that result under an energy budget, where the picture differs.

*Independent researcher. Reference implementation: <https://github.com/Anode1/SMBPANN>.

The idea: an exhaustive, tangled net $\rightarrow$ structure emerges through a few operators

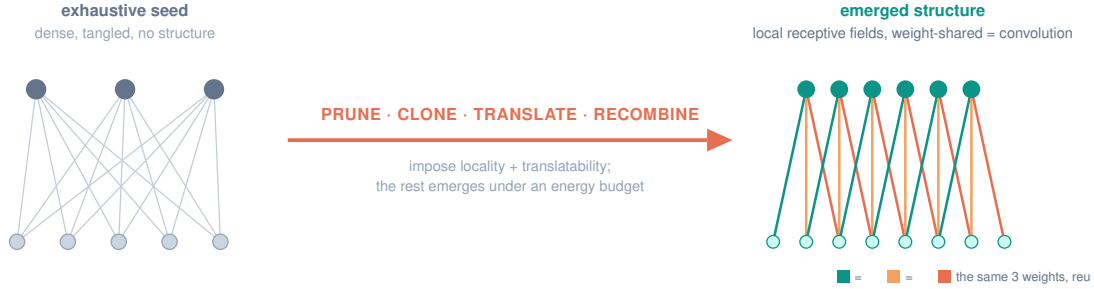


Figure 1: The idea. An exhaustive, tangled network becomes an ordered convolution through a few structural operators. We impose the domain’s real symmetries (locality, translatability); the operators, prune, clone, translate, recombine, are biological analogs we study individually and then *chain* (find a block, then clone and translate it, Section 3.4). Chaining in the fourth, where the search must *discover* the decomposition, does not cleanly emerge (Section 3.5).

Contribution. Our one mechanistic result is the sharing/energy *decoupling*: a compact filter cannot emerge under per-connection pruning, because sharing decouples connection count from parameter count, and emerges only once mutation acts on the whole shared feature (as a gene acts on every instance it builds, not on one edge). Around it we draw a *map*, using the energy GA as a *lens* rather than an optimizer, of which pieces of convolution are imposed priors and which emerge from an exhaustive seed, with the boundary where emergent composition needs supervision reported as a result rather than omitted: on a two-operation task the channels do not specialize from the composite label alone, measured against a supervised curriculum ceiling and an unshared floor, and in two dimensions the channel axis meets a capacity wall at three operations that four interventions and a heavy-compute rerun fail to move. The cost-as-lens itself has precedent: a connection cost drives modularity [11] and hierarchy [12], and Weight Agnostic Networks [13] isolate what structure alone contributes; what is new is turning it on *convolution*, to separate imposed priors from emergent pieces. The cost-driven settings carry no weight-tied parameters, so the decoupling reported here cannot arise in them: it is a property of what a mutation costs once one weight serves many positions. This map is also distinct from gradient pruning (Optimal Brain Damage; the Lottery Ticket), from DARTS’ gradient-relaxed supernet, and from NEAT’s grow-from-minimal, which augment rather than prune. Much of the rest, weight sharing is data-efficient; a depth that grows with but overshoots the receptive field, reproduces known inductive-bias facts, which we verify carefully rather than claim as new.

Method in one paragraph. A one-hidden-layer network whose connectivity is a binary mask is evolved by a genetic algorithm from an all-ones (dense) seed. Fitness is validation accuracy on a scarce training set; the objective is to minimize energy (the number of connections, or free parameters) subject to accuracy $\geq$ target. Mutation is mostly-prune, rarely-grow, an annealing schedule, in which a prune is a downhill move in energy and a rare added connection is an uphill thermal fluctuation. All numbers below are means over independent seeds; each experiment is a self-contained C99 probe, independent of the reference engine and its unit tests, reimplementing the network, plain-SGD backpropagation, and PRNG.¹

¹The same probe loop is $\sim 130\times$ slower in naive Python/NumPy than in C (measured); a compiled path (Numba, JAX) closes most of the gap, so the edge is over interpreted Python, not fundamental.

1-6 · Energy budget: dense seed → sparse, task-relevant structure

connectivity mask (rows = hidden units, columns = inputs); a filled cell = a connection



Result: feature selection emerges cleanly. Sparsity and weight-sharing also emerge; the compact aligned filter needs mutation on the shared feature (Sec. 2.3)

Figure 2: Sec. 2.1. An energy budget prunes a dense connectivity seed to a sparse mask whose surviving connections land on the task-relevant inputs. Feature selection and sparsity emerge; the compact aligned filter needs mutation on the shared feature to emerge (Sec. 2.3).

Statistics. Every comparison below is paired by seed: the two conditions see the same task and the same initial conditions, so the seed is the unit of analysis. The primary test is the Wilcoxon signed-rank test, computed against the exact null by dynamic programming where no tie in $|d|$ forces average ranks, and against the tie-corrected normal approximation otherwise; the tables mark which was used, and a p -value from the approximation is not quoted beyond 10^{-12} . Effect sizes are Hodges–Lehmann estimates, the median of the Walsh averages, with distribution-free rank intervals, computed over all pairs; the rank test drops zero differences by Wilcoxon’s convention, and both counts are reported. Paired solve-counts are binary and use McNemar’s exact test on the discordant seeds. All p -values are Holm-corrected within each experiment’s declared family. The tests are computed by `validation/pairstat.c`, a self-tested C99 tool, from the per-seed data each probe emits under `RAW=1`, and reproduce each probe’s own mean and standard error by an independent path.

2 What emerges, and what does not

2.1 Pruning: feature selection emerges; convolution does not

Under an energy budget the search prunes a dense connectivity mask to a sparse one, and on a task whose label depends on a few specific inputs it lands **90%** of the surviving connections on the task-relevant inputs (chance 0.33, no-selection drift 0.34). Feature selection, discovering *which* inputs matter, emerges cleanly (Figure 2). What does *not* emerge is the aligned local receptive field: at comparable density the surviving connections are no more in-window than chance, so the search finds *a* sparse subnetwork, not *the* convolutional one, exactly the outcome of the pruning literature (sparse subnetworks, not convolutions).

2.2 Imposing locality: a compact filter emerges, sharing is adopted

Locality is not optional; a bounded receptive field is a constraint of the problem (information is local), and every ConvNet imposes it. When we impose only that, each hidden unit sees a *contiguous* window, and a compact local field emerges directly: the width anneals to a mean of $\approx K$ (the kernel size), the compact filter the energy budget alone could not produce. A weight-tying gene (tie weights by within-window offset, i.e. translation invariance) is *adopted* by 0.89 of the population; but its accuracy benefit is within noise, and the shared arm stays wider, because shared energy charges only the maximum width, so the mean has no gradient. The pieces of convolution keep emerging separately; the clean compact *shared* whole does not dominate.

2.3 The compact filter, and the mutation that finds it

What “an energy budget” means changes once weights are shared, and that is where the mechanism of this paper sits. Removing a single connection no longer reduces the parameter count, since that offset is still used by other positions, so a mutation acting on one edge has no energy gradient to tighten the kernel. The right unit of mutation is not the edge but the shared feature.

The compact filter does emerge, under the right mutation. A genetic change is local in the genome but global in phenotypic reach: through pleiotropy the same gene is expressed at every instance of the structure it specifies (all the “similar blocks and duplicated edges”), not at one edge. So for a weight-shared filter the natural unit of mutation is the shared *offset*, all the connections that reuse that weight; flipping a whole offset removes all its duplicated edges in one move and reduces the parameter count, which restores the energy gradient a single-connection edit cannot give. Under this grouped mutation the kernel contracts: the surviving tap count falls to 2.6 (near $K = 3$) over 16 seeds, and a width penalty sharpens it to contiguity 0.80 on the generative offsets. It is not a perfect $K = 3$ kernel (span ≈ 4), and the offset genome makes the connectivity translation-invariant by construction, but the compact filter largely emerges once mutation acts on the shared mechanism, as real genetics does.

And it emerges from either direction. Run the same energy GA under grouped mutation from a *minimal* seed and grow up (a NEAT-style direction), and it reaches the same kernel within noise as the dense seed pruned down (24 seeds); the emergence does not depend on the starting point. Grow-and-prune sparse training (SET, RigL) moves both ways too, but toward accuracy and sparsity; this *path-independence* of the emergent *structure*, one objective reaching it from an exhaustive and a minimal seed alike, we treat as a robustness check, not a novelty claim.

2.4 Directed search vs. random: tidier, and modestly better

Does the *directed* energy search do real work, or would random sampling find as good a filter? The question is sharp because on a real benchmark our companion crossover study found evolution barely edges random, inside the noise. **Method.** At a *matched evaluation budget* we run the grouped-mutation GA against random offset masks (drawn across densities, kept by the same objective) over 200 *paired* seeds (same task per seed) at five input sizes N , and report both structure (contiguity) and task accuracy (Figure 3, Table 1).

Result: tidier, and modestly better. At 200 paired seeds the GA beats random on *both* axes (Table 1): a large structural (contiguity) gap, significant from $N=16$ on ($p_{\text{Holm}} \leq 2.6 \times 10^{-4}$) and indistinguishable from zero at $N=12$ ($p=0.33$), and a task-accuracy gap an order of magnitude smaller, significant at every N but with an interval that still reaches zero at the smallest one. This is what a directed optimizer should do on an objective it can climb: the large win is on the energy term it descends by mutation where random can only sample, a small accuracy gain alongside. At scale it is *less diffuse than random*, not compact (on-relevance 0.21 at $N=28$); a compact filter emerges only at $N=12$.

The edge is direction, not re-evaluation. The GA re-evaluates each survivor afresh every generation, so a marginal mask gets many attempts at the gate while random gets one. Give random $r=8$ best-of- r evaluations per mask at the same budget and the gap does not shrink, it *widens* (50 seeds): best-of- r costs random distinct masks, and exploration beats noise-averaging.

Why it matters. This *reconciles* the crossover null and confirms the founding conjecture, that a directed search beats random when the objective has structure to climb: on NAS-Bench-101 the top was flat, so directed search only barely edged random, inside the noise; here the energy objective is exploitable. This is the classical evolutionary-search picture, a directed search

N	paired contiguity gap	p_{Holm}	paired accuracy gap	p_{Holm}	win/tie/loss (contig.)
12	$+0.02 \pm 0.02$	0.33	$+0.0040 \pm 0.0018$	0.025	62/88/50
16	$+0.08 \pm 0.02$	2.6×10^{-4}	$+0.0048 \pm 0.0021$	0.025	100/44/56
20	$+0.17 \pm 0.02$	$< 10^{-12}$	$+0.0087 \pm 0.0019$	1.2×10^{-4}	146/15/39
24	$+0.13 \pm 0.02$	5.5×10^{-9}	$+0.0089 \pm 0.0020$	2.8×10^{-5}	135/8/57
28	$+0.15 \pm 0.02$	$< 10^{-12}$	$+0.0108 \pm 0.0025$	1.8×10^{-6}	156/2/42

Table 1: Directed GA vs random at matched evaluations, 200 paired seeds; Wilcoxon signed-rank, Holm-corrected across the family of 15 tests (three metrics $\times$ five N). The paired *contiguity* (structural) gap is not significant at $N=12$ ($p=0.33$, Hodges–Lehmann estimate 0) and is significant at every larger N , peaking at $N=20$ before plateauing. The paired *accuracy* (task) gap is significant at every N , but is an order of magnitude smaller and its rank interval still reaches zero at $N=12$ (HL= $+0.0030$, CI $[0.0000, 0.0060]$), so we read it as clear from $N=20$ and marginal below. The structural gap also survives a budget scaled with N and a repeated-evaluation control (random given $r=8$ best-of- r evaluations per mask at the same budget, 50 seeds, which if anything *widens* it).

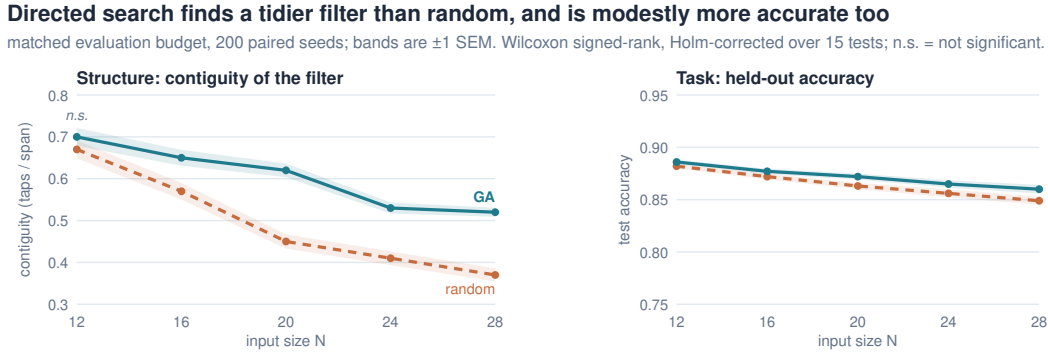


Figure 3: Directed search is *tidier*, and *modestly better*. *Left*: filter contiguity, GA vs random at matched budget, the GA is clearly more contiguous (up to 8 SEM). *Right*: held-out accuracy on the same seeds and the same axis, a small but significant GA edge (2–5 SEM). The GA wins largely on the structural term the objective rewards, and slightly on the task. 200 paired seeds, bands ± 1 SEM.

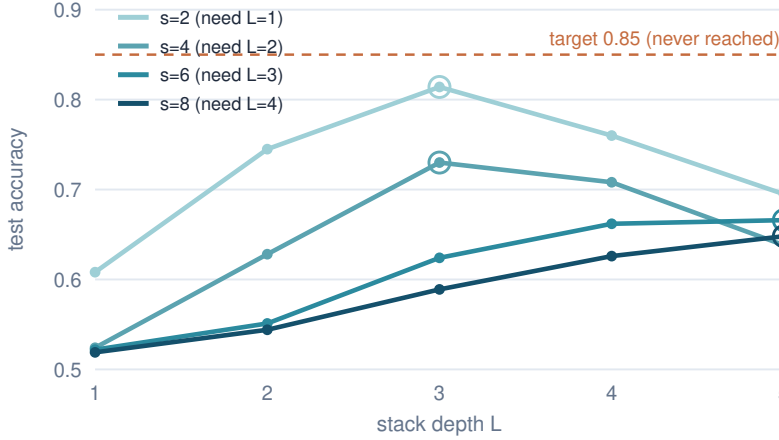
separating from random only once a gradient exists (Needle-versus-OneMax [18]), an advantage that requires exploitable structure [21, 20] and need not hold against plain hill-climbing [19]. Our claim is not the principle but its instantiation on this substrate, the energy budget supplying the gradient a flat accuracy-only objective lacks. At the sizes tested the structural gap does not keep widening; it peaks near $N=20$ and plateaus (Table 1). This plateau reflects the landscape, not the search budget: growing the budget *quadratically* in N (rather than linearly) lifts the $N=28$ structural gap by only about one SEM (100 seeds), so more compute does not reopen it. Whether it grows again at larger scale, where random covers a vanishing fraction of a space growing like 2^{2N} , remains a conjecture beyond $N \leq 28$, not a measurement.

2.5 Emergent depth grows with the receptive field, but overshoots it

Impose the feed-forward composition rule too, stack one reused block type, and the depth is the one free dimension. On a task that fires only when two spikes are exactly distance s apart (a unit must see both at once, receptive field $\geq s+1$, i.e. depth $\geq s/2$), accuracy sits near chance while the stack is too shallow to span the pair and rises with depth, and the depth of best accuracy

Emergent depth grows with the required receptive field, but overshoots it

test accuracy vs stack depth L , by required spike distance s ; fair mean over 4 restarts, 30 seeds.



Best depth (ringed) peaks at $L \approx 3, 3, 5, 5$ for $s=2, 4, 6, 8$, deeper than the required $1, 2, 3, 4$: a trend that overshoots.

Figure 4: Emergent depth grows with the required receptive field but overshoots it. Taking the mean over restarts, accuracy is near chance for stacks too shallow to span the spike pair and rises with depth, the optimum deepening with s ; but it never reaches the 0.85 target and the best depth exceeds $s/2$ (the clean target-crossing staircase was a best-of-restarts artifact).

grows with s (Figure 4). But taking the mean over restarts, the match is only *qualitative*: peak accuracy falls with s (0.81, 0.73, 0.67, 0.65 for $s=2, 4, 6, 8$) and never reaches the 0.85 target, so no depth is cleanly “selected,” and the best depth *overshoots* the required $s/2$ ($L \approx 3, 3, 5, 5$ versus $1, 2, 3, 4$). The clean target-crossing staircase we first reported was a best-of-restarts artifact; measured this way, depth is a trend with the same overshoot the channel axis shows (Sec. 3.5). The two deepest peaks ($s=6, 8$) sit at the search ceiling $L=5$, so their overshoot there is a lower bound; a larger, deeper sweep would tighten both.

3 The operators: reuse, recombination, translation

3.1 Reuse beats a free search at equal compute, up to a point

Composing blocks explodes combinatorially, so we adopt the heuristic LeCun did: reuse one block type, stacked, and search only the depth. At *equal compute* (24 seeds), reuse is the tractable near-optimum *up to a point*: enumerating uniform reused blocks ties the free heterogeneous search (a GA over arbitrary dilated-block sequences) on the easiest task ($s=4$: both 24/24, no discordant seed at all, free marginally cheaper) and again at $s=8$ (24/24 vs 22/24: only two seeds discordant, McNemar $p=0.5$, so reuse’s apparent edge there is not evidence of one). But at the hardest single-operation task the ordering inverts, and this time the test agrees: at $s=12$ the free search solves 23/24 where uniform reuse manages only 15/24, eight discordant seeds all favouring the free search ($p_{\text{Holm}}=0.023$), because a deep *uniform* stack is hard to train while a mixed-dilation sequence reaches the same span at a shallower, more trainable depth. So block diversity buys nothing while the required reach is small, and the reuse heuristic is the near-optimum there, measured rather than assumed; but its edge fades as the task deepens, foreshadowing the two-operation task below where heterogeneity becomes required. This mirrors cell-based NAS (ENAS, DARTS), which search one cell and stack it.

The operators: $A \Rightarrow B$, and exactly what each one does

structure before (grey) — operator (orange) — structure after (teal)

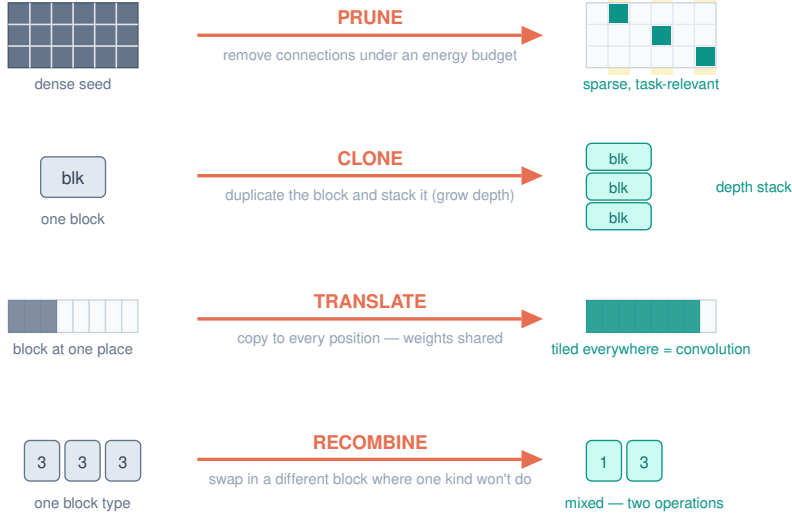


Figure 5: The four structural operators as $A \rightarrow B$, each with its exact action. Prune (remove connections under an energy budget), clone (duplicate and stack, for depth), translate (copy to every position with shared weights, a convolution), recombine (swap in a different block where one kind will not do). Each maps to a biological analog: pruning, gene/segment duplication, transposition/serial homology, sexual recombination. Of the four, only *prune* is applied inside the energy GA; *clone* and *translate* are studied as fixed shared-versus-unshared architectures (Sec. 3.1, 3.3), and *recombine* as a search over heterogeneous block sequences (mutation over sequences, not two-parent crossover).

3.2 Recombination is required when a task needs two different operations

Reuse breaks exactly where the clone-versus-recombine analogy suggests. On a task that needs a fine local motif (+, −, +) at adjacent positions (visible only to a dilation-1 block) *and* a far spike at distance s (reached cheaply only by a dilation-3 block), no single reused block serves both roles: a uniform dilation-3 stack is blind to the fine motif (solving 0/24), and the best single reused block tops out at 13/24 ($s=8$) and 5/24 ($s=12$). Recombination, a GA over mixed block sequences, solves 24/24 and 19/24 at about half the energy, with every discordant seed favouring recombination at both distances (11 and 14 of them; McNemar $p_{\text{Holm}}=2.0 \times 10^{-3}$ and 3.7×10^{-4}), and its solutions are literal two-operation mixes such as [1, 3]: one fine block, one coarse-reach block (Figure 6). Clone when scaling a working module; recombine only when you need a new kind of part, runners when the structure repeats, seeds when it must change.

3.3 Translation and scale: weight sharing is data-efficient (a reproduction)

The third operator, translation, replicating one working detector across positions with shared weights (transposition, serial homology, cortical-map tiling), is convolution’s tiling reached by replication. On a translation-invariant motif task a weight-shared filter (3 weights) beats a locally-connected baseline (one filter per position) at every training-set size, most when data is scarcest: the paired gap runs +0.24 at 16 examples to +0.14 at 128 (mean over restarts, ± 1 SD, 40 seeds), sharing winning on 40/40, 39/40, 40/40 and 40/40 seeds ($p_{\text{Holm}} \leq 7.4 \times 10^{-8}$ at every size), the textbook weight-sharing argument, a phenomenon of *sharing* not architecture

9-11 · Reuse vs recombination: clone a working block; recombine when you need a new part

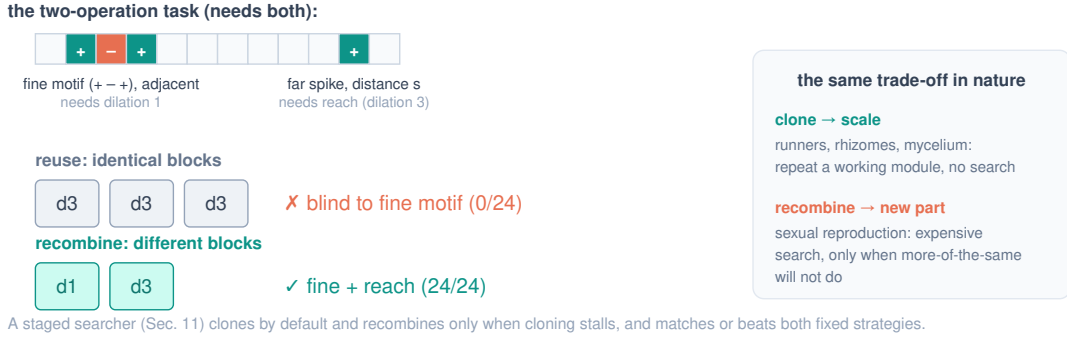


Figure 6: Reuse vs recombination, and the clone/recombine trade-off nature also makes. One reused block suffices for repetitive structure; a task needing two different operations forces recombination of different blocks.

search (both arms plain SGD). The advantage *grows with input size* (Figure 7, 250 seeds): the shared arm stays flat at ≈ 0.94 while the locally-connected baseline falls to chance and bloats to hundreds of weights; the gap widens from $+0.25$ to $+0.42$ then saturates, winning on at least 249 of 250 seeds at every size ($p_{\text{Holm}} < 10^{-12}$), and survives an oracle-supervised baseline ($+0.14$ to $+0.38$, $p_{\text{Holm}} \leq 1.0 \times 10^{-10}$, 60 seeds), so it is not a training-rule artifact.

3.4 The developmental chain: find a block, then clone and translate it

The operators above were each studied alone; here we *chain* the core of them in one run and compare a developmental trajectory against searching structure from scratch. On a translation-invariant motif task with scarce data (24 examples, 40 seeds), the trajectory is: (1) search from scratch, P independent per-position detectors trained from random; (2) find one stable shared block; (3) clone and translate it, the same filter tiled at every position (a convolution assembled by replication, not re-searched); (4) refine in place. Search-from-scratch *starves*, each position sees few examples, and reaches only 0.60; finding one block and tiling it reaches 0.86 (paired $+0.249$, all 40/40 seeds, Wilcoxon $p_{\text{Holm}} = 5.5 \times 10^{-12}$). The $+0.25$ is the *same* weight-sharing data-efficiency effect of Sec. 3.3 (one tiled block learns from every position while the P independent per-position detectors starve), now shown inside a chained pipeline, not a separate “reuse beats re-search” mechanism, and a *shared* baseline trained end-to-end on the same label captures most of it. What the chain adds is the two side findings: refining the tiled copies in place *hurts* (0.84), because independent fine-tuning re-introduces per-position starvation and breaks the sharing that made reuse work; and annealing jitter hurts more (0.76), because reuse lands at the optimum, so perturbation only kicks it off, jitter is for escaping bad minima and reuse avoids them. Both side findings are small but consistent across seeds: refinement costs -0.019 (CI $[-0.024, -0.012]$) on 38 of 40 seeds ($p_{\text{Holm}} = 6.8 \times 10^{-8}$), jitter costs -0.105 on all 40 ($p_{\text{Holm}} = 5.5 \times 10^{-12}$). Their unpaired standard deviations overlap, so neither is visible without pairing. This chains find, clone, and translate; whether the fourth operator, recombination of a *second* discovered block, *emerges* when the search must find the decomposition itself is the question of Sec. 3.5.

3.5 Does composition emerge when the search must find it? A boundary

The chain above, like the reuse and translation results, hands the search a decomposition and asks it to assemble the pieces. The sharper question is whether the *search itself*, given only the composite label and an energy budget, discovers the decomposition, both how many parts and

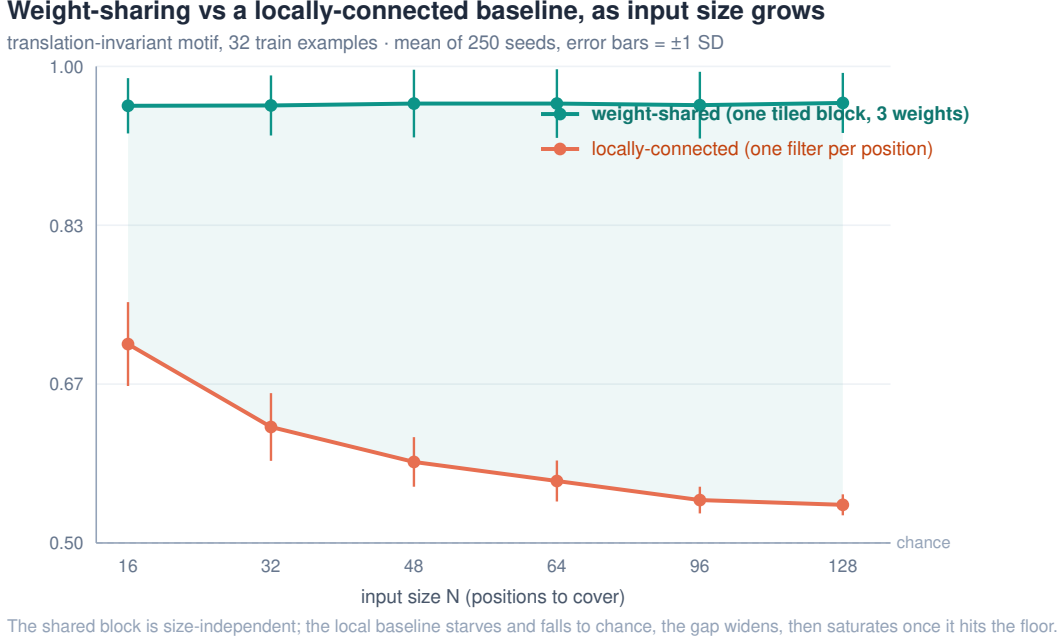


Figure 7: The weight-sharing advantage as input size grows (mean of 250 seeds, error bars ± 1 SD). The shared block is size-independent; the locally-connected baseline starves and falls to chance. The gap widens, then saturates once the baseline hits the floor.

which. We test it on a two-operation task (motif *A* and *B*, each at a random position; positive iff both present), where one weight-shared tiled channel caps at 0.75 by construction: sweep the channel count C under an energy budget (the energy-selection of Sec. 2.5, now on channels; write C^* for the count it selects), train each candidate’s shared filters and read-out jointly on the composite label *alone*, against an auxiliary-label *curriculum* (find *A*, find *B*, freeze, combine) as a supervised ceiling and an unshared per-position search as the floor (40 paired seeds).

Result: composition does not cleanly emerge without supervision. A one-operation task selects $C^*=1$; but on the two-operation task the two channels specialize to the two motifs in only 55% of runs (the rest collapse onto one), so $C=2$ plateaus at 0.81 (a shared baseline), the curriculum reaches 0.87, and the energy-selected count *overshoots* to $C^*=3$. The supervised curriculum’s margin over the discovered solution is $+0.062$ on 32 of 40 seeds ($p_{\text{Holm}}=4.9\times 10^{-4}$), while the discovered solution does clear the unshared floor by $+0.161$ on all 40 ($p_{\text{Holm}}=3.6\times 10^{-12}$): the search finds sharing but not the decomposition. Directed search under an energy budget does not, on its own, find the two-part decomposition, it under-specializes and pays for a redundant channel; the clean split appears only with per-operation supervision. This sharpens the contrast between the axes: emergent *depth* at least grows with the receptive field from the task label alone (Sec. 2.5, even if it overshoots), whereas emergent *composition* on the channel axis does not organize at all without per-operation supervision. Section 4 carries the same axis into two dimensions, where the boundary has a locatable ceiling. It also explains a chain that only *appears* to recombine, hand it the decomposition and its gain is weight sharing plus that supervision, not emergent composition.

3.6 Toward audio: a periodic task

Audio adds long-range *periodicity* (pitch) on top of local detail, a natural test of whether the reuse-versus-recombination result survives. Replacing the recombination task’s far spike with a periodic spike train at period p (the pitch analog) keeps the two-operation structure and reproduces the finding: at the decisive period, recombination solves 100/100 where the best

reused block reaches 93–94/100 and a fine-only clone collapses to 14–87/100, at lower depth and with heterogeneous filters like [1, 4] (100 seeds). The boundary is *soft*, though: a well-chosen uniform dilation still solves most cases (a coarse block, scanned and max-pooled, partially recovers the fine motif), so recombination’s edge here is real but modest.

4 The boundary in 2-D

The composition boundary of Sec. 3.5 was measured on the channel axis in one dimension, where a “channel” is an abstraction of the task. In two dimensions the axis has a natural reading, and the boundary turns out to have a locatable ceiling.

In two dimensions “two different operations” means two feature *channels*, a horizontal and a vertical filter, since one channel is one orientation. A task requiring both orientations present caps a single channel near 0.75 (the analytic ceiling a lone orientation detector can reach), and accuracy rises with channels ($0.69 \rightarrow 0.77 \rightarrow 0.86 \rightarrow 0.89$ over $C=1..4$, 24 seeds): a gradual rise, not a sharp $C=2$ jump, but the load-bearing fact, one channel is not enough for two operations, is significant. The tidy generalization “channel count = operation count,” however, **fails**: growing channels to solve a K -operation task gives selected $C^* \approx 1.2, 3.2, 5.0$ for $K=1, 2, 3$, so $K=2$ already overshoots and $K=3$ breaks, sub-target at every channel count ($C=5$: 0.79), only 2 of 24 seeds solving.

This $K=3$ wall is genuine, not under-training. Four interventions fail to move it: more channels, a nonlinear (MLP) readout, competition (lateral inhibition, the mechanism that self-organizes cortical orientation columns), and, with a budget caveat, a deeper extractor. An independent heavy-compute rerun ($2.5\times$ epochs, $2\times$ signal) leaves $K=3$ sub-target at every channel count, confirming the wall is structural for the channel axis. Per-channel detection error compounds multiplicatively ($0.95^3 \approx 0.86$, at the target edge), and no cheap refinement moves a multiplicative wall; real vision clears such tasks with scale (depth, normalization, residuals, vast data), not one clever pressure. We report this boundary plainly rather than stopping at small K where the staircase still looked clean.

5 Related work, and how we differ

The closest neighbors, and the distinction: **DARTS** starts from an exhaustive supernet and prunes to a final architecture, closest in spirit, but via continuous gradient relaxation over pre-defined operations, not an energy GA, and it never asks whether convolution emerges. **Optimal Brain Damage** and the **Lottery Ticket Hypothesis** prune dense to sparse, but by gradient magnitude, and find sparse subnetworks, not convolutions. Most directly, convolutional structure has been shown to *emerge* in fully-connected networks trained by gradient descent [8] and learned from scratch by regularization [9]; those results obtain locality and translation structure *without imposing* them. We impose the domain symmetries and ask, under a *non-gradient* evolutionary energy budget that pays per connection, which of the remaining pieces the pressure yields on its own and which it does not: a complementary, more scaffolded probe, not the same emergence. Which of convolution’s pieces carries the advantage, locality or weight sharing, is itself a studied decomposition [15, 17], generally finding locality the dominant bias, and sparse local connectivity has since been shown to give non-vacuous generalization bounds in a regime where fully-connected networks provably fail [16] (added September 2026); those results impose the sparsity, where we ask whether it emerges. We add the emergence-under-evolution axis and the composition boundary to that decomposition. **NEAT**, **HyperNEAT**, and Cellular Encoding evolve topology but *grow from a minimal seed*, the opposite direction to pruning from exhaustive. **Cell-based NAS** (ENAS, regularized evolution) reuses one cell and stacks it, which our reuse observations echo. The essential difference is the *use* of the GA: evolutionary NAS

runs it as an *optimizer* to find a performant architecture, whereas we run it as a *lens* to characterize which pieces of convolution emerge, and which refuse, under an energy budget from an exhaustive seed. Using an evolutionary connection cost as a lens on emergent *structure* is not itself new: a connection cost drives *modularity* [11] and *hierarchy* [12] to emerge, and Weight Agnostic Networks [13] evolve topology to isolate what structure alone contributes. What we are not aware of elsewhere is this cost-as-lens applied to decomposing *convolution* into imposed-versus-emergent pieces, with the sharing/energy decoupling mechanism. That characterization, and its negatives, is the claim. The sharing/energy decoupling, that a compact shared filter must be edited at feature granularity rather than one connection at a time, shares an intuition with *structured* versus unstructured pruning [10] but not a result: that work removes whole filters from a trained ConvNet to cut computation and storage, and makes no claim about search or emergence. Here the convolution does not yet exist, and the granularity of mutation decides whether it can appear at all.

Two framings place this experiment in a longer line. The view of network topology as problem-specific inductive bias that must be engineered by hand goes back decades [25]; here we ask the complementary question, whether that structure can instead *emerge* from the energy budget rather than be built in. Viewed more broadly, the energy budget is a description-length pressure: the emergent weight-shared filter is a compressor whose inductive bias, locality and sharing, matches the task’s translation symmetry, a connection developed further in [24]. It also continues the efficient-coding tradition [14], where localized receptive fields emerge under a sparseness/energy budget, except that there locality itself emerges whereas here it is imposed.

6 Limitations

The tasks are small synthetic constructions built so the intended structure is optimal; they demonstrate what emerges under an energy budget on a task shaped like convolution, and do not by themselves generalize to real data. The reuse and weight-sharing results are a property of *fixed* shared-versus-unshared architectures (data efficiency under SGD), not evidence about what the evolutionary search discovers; we label them accordingly. The developmental chain (Section 3.4) covers find, clone, and translate; when the search must *discover* the recombination of a second block it does not cleanly do so (Sec. 3.5). What limits this work is scale, not the absence of an idea: the new parts, the sharing/energy decoupling and the directed-vs-random reconciliation, are real but small, and the tasks are toy. But this method is strongest at *characterization*, the maps and boundaries a performance-driven search leaves unmeasured, rather than at scale.

7 Conclusion

Impose the domain’s real symmetries; a few biological operators, studied here one at a time, could then assemble the rest. Under an energy budget on an exhaustive seed, sparsity and feature selection emerge and weight sharing is adopted, and a compact convolution emerges too, but only once mutation acts on the shared feature rather than the single edge, for a decoupling reason we can state in a line. On top of imposed priors, depth grows with the receptive field (overshooting it), reuse beats a free search at equal compute up to moderate depth (a heterogeneous search overtakes it on the deepest single-operation task), recombination is required outright once a task needs a different operation, and weight-shared tiling is data-efficient at a degree that grows with scale then saturates. Where the search must *discover* a compositional decomposition itself it does not, the parts failing to specialize without supervision (Sec. 3.5), a boundary we mark rather than paper over. The contribution is the map, the negatives, and the reproducibility; each experiment states its seed count and regenerates from a named probe in the reference

implementation, listed in the appendix.

Future work. The sharpest next step is the *minimal-supervision boundary* of the composition negative (Sec. 3.5): sweep supervision from none upward for the threshold, if any, at which the parts snap into place, a phase transition this setup is built to measure. A second is *real* audio, where a first spoken-digit attempt found random matching the directed search. That attempt is inconclusive rather than negative: the framing pipeline does not reach usable accuracy on the digit pair (every filter support we tested lands between 0.54 and 0.59 held out, and a thirtyfold increase in training epochs does not move it), so the outcome licenses no claim about the shape of the optimal filter. A clean test needs a front end on which a known-good classifier works, and is future work.

Appendix: how each experiment regenerates

Every experiment is a self-contained C99 probe in `validation/` of the reference implementation [23], built and run as `make probe && ./probe`. Seed counts are stated in each section; where a run departs from a probe’s compiled default, the flags are given below. Each flag is equivalently settable through its upper-case environment variable, so `--seeds 200` and `SEEDS=200` are interchangeable.

References

- [1] Y. LeCun, J. S. Denker, S. A. Solla. Optimal Brain Damage. *NeurIPS* 1990.
- [2] Y. LeCun et al. Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation* 1(4), 1989.
- [3] J. Frankle, M. Carbin. The Lottery Ticket Hypothesis. *ICLR* 2019.
- [4] H. Liu, K. Simonyan, Y. Yang. DARTS: Differentiable Architecture Search. *ICLR* 2019.
- [5] H. Pham, M. Guan, B. Zoph, Q. Le, J. Dean. Efficient Neural Architecture Search via Parameter Sharing. *ICML* 2018.
- [6] K. O. Stanley, R. Miikkulainen. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10(2), 2002.
- [7] E. Real, A. Aggarwal, Y. Huang, Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI* 2019.
- [8] A. Ingrosso, S. Goldt. Data-driven emergence of convolutional structure in neural networks. *PNAS* 119(40), 2022.
- [9] B. Neyshabur. Towards Learning Convolutions from Scratch. *NeurIPS* 2020.
- [10] H. Li, A. Kadav, I. Durdanovic, H. Samet, H. P. Graf. Pruning Filters for Efficient ConvNets. *ICLR* 2017.
- [11] J. Clune, J.-B. Mouret, H. Lipson. The evolutionary origins of modularity. *Proc. R. Soc. B* 280(1755), 2013.
- [12] H. Mengistu, J. Huizinga, J.-B. Mouret, J. Clune. The Evolutionary Origins of Hierarchy. *PLOS Comput. Biol.* 12(6), 2016.
- [13] A. Gaier, D. Ha. Weight Agnostic Neural Networks. *NeurIPS* 2019.
- [14] B. A. Olshausen, D. J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature* 381, 1996.
- [15] Z. Wang, L. Wu. Theoretical Analysis of Inductive Biases in Deep Convolutional Networks. *NeurIPS* 2023.
- [16] T. Liang, E. Singh, R. Parhi, A. Cloninger, Y.-X. Wang. Does Sparse Connectivity Improve Generalization? Convolutional Networks Below the Edge of Stability. arXiv:2603.04807, 2026.

Section	Probe	Run
2.1 pruning, feature selection	<code>conv_emerge</code>	defaults
2.2 locality imposed	<code>emerge_local</code>	defaults
2.2 weight-tying gene adopted	<code>emerge_tie</code>	defaults
2.3 compact filter, grouped mutation	<code>emerge_offset</code>	defaults
2.3 same kernel from a minimal seed	<code>emerge_minimal</code>	defaults
2.4 directed vs. random	<code>emerge_prove</code>	--seeds 200, archived as <code>scratch_emerge_prove_200.out</code> ; the three control regimes of Sec. 2.4 are the fixed, scaled and <code>denoise_r8</code> (--revals 8) blocks in <code>scratch_prove_sweep.out</code>
2.5 emergent depth	<code>emerge_compose</code>	defaults
3.1 reuse vs. free search	<code>emerge_arch</code>	defaults
3.2 recombination, two operations	<code>emerge_twoop</code>	defaults
3.3 translation, weight sharing	<code>emerge_translate</code>	defaults
3.3 scale sweep	<code>emerge_scale</code>	defaults
3.3 oracle-supervised baseline	<code>emerge_baseline</code>	defaults
3.4 developmental chain	<code>emerge_develop</code>	defaults
3.5 composition boundary	<code>emerge_discover</code>	defaults
3.6 periodic (audio) task	<code>emerge_pitch</code>	--seeds 100
4 channels for two operations	<code>emerge_2d_chan</code>	defaults
4 emergent channel count vs K	<code>emerge_2d_grow</code>	defaults
4 intervention: combining layer	<code>emerge_2d_deep</code>	defaults
4 intervention: deeper extractor	<code>emerge_2d_deep2</code>	defaults
4 intervention: lateral inhibition	<code>emerge_2d_compete</code>	defaults
all paired tests	<code>pairstat</code>	each probe above re-run with <code>RAW=1</code> emits one line per seed; piping those to <code>pairstat</code> prints the tests of the Statistics paragraph. <code>./pairstat --selftest</code> checks it against hand-computable cases. <code>scripts/run_pairstats.sh</code> runs the whole set

- [17] A. Lahoti, S. Karp, E. Winston, A. Singh, Y. Li. Role of Locality and Weight Sharing in Image-Based Tasks: A Sample Complexity Separation between CNNs, LCNs, and FCNs. arXiv:2403.15707, 2024.
- [18] S. Droste, T. Jansen, I. Wegener. On the analysis of the (1+1) evolutionary algorithm. *Theoretical Computer Science* 276, 2002.
- [19] M. Mitchell, J. H. Holland, S. Forrest. When Will a Genetic Algorithm Outperform Hill Climbing? *NeurIPS* 1993.
- [20] T. Jones, S. Forrest. Fitness Distance Correlation as a Measure of Problem Difficulty for Genetic Algorithms. *ICGA* 1995.
- [21] D. H. Wolpert, W. G. Macready. No Free Lunch Theorems for Optimization. *IEEE Trans. Evolutionary Computation* 1(1), 1997.
- [22] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi. Optimization by Simulated Annealing. *Science* 220, 1983.
- [23] V. Gavrilov. SMBPANN reference implementation, 2001–2015–2026. <https://github.com/Anode1/SMBPANN>.
- [24] V. Gavrilov. Intelligence Is the Discovery of Compressors. 2026. <https://doi.org/10.5281/zenodo.20440110>.

- [25] V. Gavrilov. Backpropagation Feed-Forward Neural Networks. Seminar thesis, Open University of Israel, 1997 (digitized 2026). <https://doi.org/10.5281/zenodo.20450525>.